AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY

Amrita School of AI
COIMBATORE CAMPUS

EXERCISE 08 · UNIT 2

Parallel histogram — contention and false sharing

An exercise with no correctness difficulty at all. Every approach gives the right answer, and the two obvious ones are slower than serial, for two different reasons.

Prepared by **Dr. Abhijith Anandakrishnan**
Assistant Professor, Amrita School of AI

EXERCISE	UNIT
ex08-openmp-histogram	2 — OpenMP and MPI
LANGUAGE	MARKS
C, OpenMP	12
SUBMIT AS	OFFERING
<ROLLNO>.c	Odd Semester 2026-27

The problem

Count how many elements of an array fall into each of 16 equal-width bins:

```
void histogram(const double *a, size_t n, double lo, double hi, long *bins);
```

Values below `lo` go in bin 0, values at or above `hi` go in bin 15, and `bins[]` is zeroed by the function before counting.

The serial version is four lines and is given to you. Your job is to make it **scale**.

> **NOTE · THIS EXERCISE IS GRADED MOSTLY ON SPEED**
>
> Correctness here is easy: almost anything you write will produce the right counts. Eight of the twelve marks are for scaling and for the memory behaviour that produces it. That is deliberate. In real parallel work, getting the right answer is the beginning of the problem, not the end.

The two traps

Trap 1: guard the shared array

```
#pragma omp parallel for                    /* SLOW */
for (size_t i = 0; i < n; i++)
#pragma omp atomic
    bins[bin_of(a[i])]++;
```

Perfectly correct. Every thread agrees on the answer, every time.

Measured on a two-core machine, this runs at **0.36x**, that is nearly three times *slower* than the serial version. Every single element pays for an atomic read-modify-write, and with only 16 bins the threads are constantly contending for the same handful of addresses. You have added synchronisation to every iteration and bought nothing.

`#pragma omp critical` in the same place is worse still.

Trap 2: give each thread a row of a shared array

```
long tbins[nthreads][NBINS];                /* 16 longs = 128 bytes per thread */
```

No atomics. No race. Each thread touches only its own row. And it is often still disappointing.

128 bytes is two cache lines, so **thread 0's last bins and thread 1's first bins sit on the same 64-byte line**. Every increment by either thread invalidates that line in the other core's cache, and the line ping-pongs across the interconnect.

DEFINITION · FALSE SHARING

A performance collapse caused by several cores writing to *different* variables that happen to share one cache line. There is no data race and no wrong answer, which is exactly what makes it hard to find: the program is correct, and slow, and nothing in the source looks wrong.

The tell-tale symptom is that it gets **worse** as you add threads.

What actually works

Accumulate into storage that is private to the thread and cannot share a line with anyone else's, then combine the per-thread results **once** at the end.

There is more than one way to arrange that. One is to declare the accumulator inside the parallel region so it lives on the thread's own stack:

```
#pragma omp parallel
{
    long local[NBINS];                  /* on this thread's stack */
    for (int i = 0; i < NBINS; i++) local[i] = 0;

#pragma omp for schedule(static)
    for (size_t i = 0; i < n; i++) local[bin_of(a[i])]++;

#pragma omp critical                    /* 16 updates per THREAD, not per element */
    for (int i = 0; i < NBINS; i++) bins[i] += local[i];
}
```

Different threads' stacks are pages apart, so false sharing is impossible by construction rather than by careful padding arithmetic.

Note where the `critical` is. Inside the element loop it would be a disaster; after the loop it runs once per thread, so on 96 threads it costs 96 short critical sections rather than four million.

BEYOND THE SYLLABUS · THE MODERN ONE-LINER

OpenMP 4.5 and later can reduce over an array section directly:

```c
#pragma omp parallel for reduction(+ : bins[:NBINS])
for (size_t i = 0; i < n; i++) bins[bin_of(a[i])]++;
```

The runtime creates the private copies and combines them for you. This is accepted, and worth knowing, but implement it the explicit way at least once first: the point of the exercise is understanding *what* the runtime is doing on your behalf and why it matters.

PITFALL · PADDING IS A FIX, NOT THE FIX

You can make the shared 2-D array work by padding each row to a whole number of cache lines. It is correct and it is what you must do when the accumulator genuinely has to be shared. But it hard-codes an assumption about cache line size, and a private accumulator avoids the problem entirely.

Building and running

```
./selfcheck.sh
OMP_NUM_THREADS=8 ./selfcheck.sh
```

ON ASAICOMPUTE · SEE THE SCALING PROPERLY

Two cores cannot show you much. On the cluster:

`bash for t in 1 2 4 8 16 32; do srun --cpus-per-task=32 --time=00:05:00 \ bash -c "OMP_NUM_THREADS=$t OMP_PROC_BIND=close ./bench 20000000 20" done`

Do this for all three versions: atomic, unpadded per-thread rows, and private accumulators. Plot all three. The atomic curve goes **down** as you add threads, the unpadded curve flattens early, and only the private version scales. That plot is worth more to your understanding than this whole page of prose.

How this is marked

CHECK	MARKS	WHAT IT MEANS
Compiles cleanly	1	Builds with `-Wall -Wextra -Werror`
Uses OpenMP	1	A parallel directive is present
Public tests	2	Correctness, including the clamping rules
Hidden tests	2	Edge cases, prime lengths, conservation of the total
Deterministic across thread counts	2	Identical bins over repeated runs at 1, 2 and 4 threads
Scales against the serial reference	4	Parallel efficiency of at least 55% of the core count

Note that the marking deliberately does **not** grep your source for `atomic` or `critical`. Whether your approach serialises is decided by **measurement**, because a rule that banned the keyword outright would also reject the perfectly good combine-once-at-the-end pattern shown above.

Speedup is graded as efficiency scaled to the physical core count, so the target adapts to whatever machine grading runs on.

Submitting

```
AIE23001.c
```

One file, no `main()`.

> **NOTE · BEFORE YOU SUBMIT**
>
> Time your version against the serial one at two or more thread counts. If it is not faster, you have found trap 1 or trap 2, and the grader will find the same thing.